# >>> pràctica PA1

## >>> ‘three in a row’

Paula Justo Gili
Nadia Fernández Dominguez

# ÍNDEX:

**1. REPARTIMENT DE RESPONSABILITATS** **3**

1.1. Versió bàsica 3

1.2. Versió ampliada 8

i) abs_board_h 8

ii) main_txt 10

iii) main_gui 14

**2. DIFICULTATS PRINCIPALS** **15**

**3. OBJECTIUS ASSOLITS** **16**

**4. VALORACIÓ DE L'APRENENTATGE** **17**

# 1. REPARTIMENT DE RESPONSABILITATS

Nosaltres, com a equip, vam considerar que el més òptim per realitzar la pràctica era treballar de forma conjunta, ja que d'aquesta forma podíem compartir idees i coneixements per tal de poder obtenir resultats més complets i ben elaborats. A més, el fet de treballar juntes durant tot el desenvolupament del programa ha facilitat la resolució d'obstacles i problemes que van anar sortint al llarg de la pràctica (problemes explicats en l'apartat 2. d'aquest informe).

## 1.1. Versió bàsica

*abs_board_h*

***set_board_up(ST_PLAYER = 4):***

```
def set_board_up(ST_PLAYER = 4):
    tablero=['  ']*BSIZ**2
    cont=0
    m=0
```

Dins d'aquesta funció, a més de definir les funcions locals, hem hagut d'inicialitzar algunes variables. En primer lloc, hem definit 'tablero' com una llista on es troben totes les caselles del taulell (que estan en funció de la constant BSIZ). Cada casella del taulell, en un principi, està formada per una string buida (' ') que més endavant serà substituida per una pedra ('X ' o 'O ' depenent del jugador). En segon lloc, hem inicialitzat la variable 'cont' a 0 amb l'objectiu de saber la quantitat de moviments jugats, per tant, saber també quin jugador ha de tirar (depenent de si 'cont' és parell o senar). Per últim, hem inicialitzat la variable 'm' a 0 per tal de saber l'índex de la posició de la llista de la pedra que seleccionem.

***draw_txt(end=False):***

```
def draw_txt(end=False):
    print("","-"*(6*BSIZ-1))
    for fila in range(0,BSIZ):
        print("|", end="")
        for columna in range(0,BSIZ):
            x = fila*BSIZ + columna
            print(" ", tablero[x], "|", end="")
        print("\n","-"*(6*BSIZ-1))
```

Per fer aquesta funció, la Nadia va veure que seria bastant ràpid implementar-la utilitzant dos iteracions amb 'for', una per les files i l'altre per les columnes. Primer de tot, s'havia de escriure la línia superior del taulell fora del bucle. Després, quan ens introduïm al bucle de les files, s'ha d'escriure la primera línia vertical del taulell i fem un altre bucle per les columnes. Dins d'aquest, inicialitzem x com la posició del taulell en la que ens trobem, que dintre de la nostra llista *tablero* correspon a *fila * BSIZ + columna*, i la escrivim al terminal

seguida d'una altra línia vertical per separar la posició de la següent. Vam veure que, després de cada 'print' es feia un enter predeterminat i la Paula ho va solucionar afegint *end=""* al final d'aquestes funcions. Després de que s'hagin escrit totes les columnes de la fila, s'acaba la iteració de les files escrivint un enter amb una altra línia horitzontal i es pasa a la següent fila. Com que volíem que es pogués canviar fàcilment la dimensió del taulell, la mida d'aquest havia de ser proporcional a *BSIZ*, per tant, fent proves la Paula va trobar com havien de ser de llargues les línies horitzontals: 6 * BSIZ -1.

### `move_st(i,j):`

```
def move_st(i, j):
    k=i*BSIZ+j
    nonlocal m, cont
    g=False
    f=False
    if 0<= i < BSIZ and 0<= j < BSIZ:
        if tablero[k]=='  ':
            tablero[k]='X ' if cont%2==0 else 'O '
            cont+=1
            if cont-1>=ST_PLAYER*2:
                tablero[m]='  '
        else:
            print('This square is occupied')
            g = True
    else:
        print('This square is out of board')
        f = True

    return (cont//2<ST_PLAYER or f or g), 0 if cont%2==0 else 1 , end()
```

Primer de tot, vam utilitzar 'nonlocal' amb *m* i *cont* per tenir aquestes variables que no són de la funció local. Recordem que *cont* conté el número de tirades fetes en total i *m* sería l'índex de la llista *tablero* on es troba la pedra seleccionada (si encara no s'han llençat totes les pedres, no s'hauria seleccionat cap pedra i aquesta variable no s'utilitzaria). Després vam crear la variable *k* que correspon a la posició de la llista on volem posar la pedra. A la Nadia, se li va ocórrer crear també dues variables més, *g* i *f*, que corresponen a un booleà: *g* és 'True' si el lloc on vols moure la pedra està ocupat i *f* és 'True' si el lloc on vols moure la pedra es troba fora del taulell; que en un primer moment comencen ser 'False' les dues. Un cop creades aquestes variables locals, la Paula va pensar en primer comprovar si la casella es troba dins del taulell (*0<=i<BSIZ and 0<=j<BSIZ*). Si això no es complia, s'escrivia *This square is out of board* i *f* passava a ser 'True'. En cas contrari, s'havia de comprovar si la casella estaba buida. Si això no es complia s'escrivia *This square is occupied* i *g* passava a ser 'True'. Si la casella sí estaba buida, afegíem 'X ' o 'O ', depenen del torn de qui fos, a *tablero* al lloc *k* i sumem 1 a *cont*. Ara, la Paula va pensar que per comprovar si s'havia d'utilitzar m, és a dir, si s'havia seleccionat una pedra per canviar-la de posició, havíem de saber si *cont-1 >= ST_PLAYER * 2*. En aquest cas, la posició *m* a *tablero* passava a ser '  '.

Finalment, la funció havia de retornar:

- **stone_selected**: un booleà per saber si seguir afegint pedres a la funció *move_st* (True) o passar a seleccionar-ne amb la funció *select_st* (False). Per tant, stone_selected era 'True' quan s'havien els

jugadors no havien tirat totes les seves pedres (*cont//2 < ST_PLAYER*) o quan havies seleccionat malament la casella (*f or g*).

- **curr_player**: Havia de retornar 0 si li tocaba al primer jugador o 1 si li tocava al segon. Per tant: *0 if cont%2 == 0 else 1*.
- **end**: Havia de retornar 'True' si ha acabat el joc, és a dir, algú ha fet tres en ratlla; sinó retorna 'False'. Per tant, vam posar la funció *end()*, que és la funció que ho comprova.

### `select_st(i,j):`

```
def select_st(i, j):
    nonlocal m, cont
    m=i*BSIZ+j
    if 0<= i < BSIZ and 0<= j < BSIZ:
        if (cont%2==0 and tablero[m]=='X ') or (cont%2!=0 and tablero[m]=='O '):
            return True
        else:
            print("This square hasn't got any of your stones")
    else:
        print('This square is out of board')
```

Primer de tot, vam utilitzar un altre cop 'nonlocal' amb *m* i *cont* per tenir aquestes variables que no són de la funció local. Després vam assignar a la variable *m* la posició de tablero de la pedra que s'acaba de seleccionar. Igual que vam fer a *move_st*, vam comprovar que la casella es trobés dins del taulell (*0<=i<BSIZ and 0<=j<BSIZ*). Si això no es complia, s'escrivia *This square is out of board.* En cas contrari, s'havia de comprovar si la casella contenia una peça del jugador que li tocava tirar. Si això no es complia s'escrivia *This square hasn't got any of your stones*. Si la casella sí contenia una de les seves peces llavors la funció retornava 'True' perquè el jugador ja havía escollit una pedra correctament.

### `end():`

```
def end():
    fila = 0
    columna = 0
    diagonal1 = 0
    diagonal2 = 0
    while fila<BSIZ**2-2:
        while (fila+2)%BSIZ!=0:
            if tablero[fila]==tablero[fila+1]==tablero[fila+2]==('X ') or tablero[fila]==tablero[fila+1]==tablero[fila+2]==('O '):
                return True
            fila+=1
        fila+=2

    while columna<BSIZ*(BSIZ-2):
        if tablero[columna]==tablero[columna+BSIZ]==tablero[columna+BSIZ*2]==('X ') or tablero[columna]==tablero[columna+BSIZ]==tablero[columna+BSIZ*2]==('O '):
            return True
        columna+=1

    while diagonal1 <= BSIZ*(BSIZ-3)+BSIZ-3:
        while (diagonal1 + 2*BSIZ + 2)%BSIZ!= 0:
            if tablero[diagonal1]==tablero[BSIZ+1+diagonal1]==tablero[2*BSIZ+2+diagonal1]==('X ') or tablero[diagonal1]==tablero[BSIZ+1+diagonal1]==tablero[2*BSIZ+2+diagonal1]==('O '):
                return True
            diagonal1 += 1
        diagonal1 += 2

    while diagonal2 < BSIZ*(BSIZ-2):
        while diagonal2%BSIZ!= 0:
            if tablero[diagonal2]==tablero[diagonal2 +BSIZ -1]==tablero[diagonal2 + 2*BSIZ -2]==('X ') or tablero[diagonal2]==tablero[diagonal2 +BSIZ -1]==tablero[diagonal2 + 2*BSIZ -2]==('O '):
                return True
            diagonal2 += 1
        diagonal2 += 2

    return False
```

En primer lloc, vam inicialitzar 4 variables a 0, les quals vam utilitzar per recórrer la llista de tablero buscant tres en ratlla: *fila*, *columna*, *diagonal1* i *diagonal2*. Vam dividir aquesta funció en quatre parts i cadascuna de nosaltres va fer dues:

1. La primera part, feta per la Paula, consistia en comprovar si hi havia tres en ratlla horitzontal. En aquesta part, es recorrien tots els elements del taulell excepte els dos últims de cada fila (ja que els dos següents elements de la llista no estarien a la mateixa fila) comprovant si aquest i els dos següents tenen pedres del mateix jugador. Cada cop que la variable *fila* arriba al penúltim element de les files(*(fila + 2) % BSIZ == 0*), se li sumen 2 per passar a la següent. En el cas de que en algun element de *tablero* dels que recorrem i els dos següents tinguin pedres del mateix jugador, el funció retorna 'True'.
2. La segona part, feta també per la Paula, consistia en comprovar si hi havia tres en ratlla vertical. En aquesta part, es recorrien tots els elements del taulell excepte les dues últimes files (ja que aquestes no tenen dos elements a sota). Per tant, el bucle 'while' havia d'arribar fins a (BSIZ * BSIZ - 2). En cada interació, s'havia de comprovar si l'element *tablero[columna]* i els dos de sota eren iguals. En cas afirmatiu, un jugador haria fet tres en ratlla i la funció retornaria 'True'.
3. La tercera part, feta per la Nadia, consistia en comprovar si hi havia tres en ratlla en una diagonal decreixent. En aquesta part, es recorrien tots els elements del taulell excepte les dues últimes files i les dues últimes columnes (ja que aquestes no tenen dos elements dues posicions cap a la dreta i dues posicions cap a sota). Així doncs, el bucle 'while' havia d'arribar fins a *BSIZ * (BSIZ - 3) + BSIZ -3* i cada cop que la variable *diagonal1* arriba al penúltim element de les files (*(diagonal1 + 2) % BSIZ == 0*), se li sumen 2 per passar a la següent. En cada iteració del bucle, s'havia de comprovar si *tablero[diagonal1]*, *tablero[diagonal1 + BSIZ + 1]* i *tablero[diagonal1 + 2*BSIZ + 2]* eren el mateix element. En cas afirmatiu, un jugador hauria fet tres en ratlla i la funció retornaria 'True'.
4. L'última part, feta també per la Nadia, consistia en comprovar si hi havia tres en ratlla en una diagonal creixent. En aquesta part, es recorrien tots els elements del taulell excepte les dues primeres files i les dues últimes columnes (ja que aquestes no tenen dos elements dues posicions cap a la dreta i dues posicions cap a dalt). Així doncs, el bucle 'while' havia d'arribar fins a *BSIZ * BSIZ -2* i cada cop que la variable *diagonal2* arriba al penúltim element de les files (*(diagonal2 + 2) % BSIZ == 0*), se li sumen 2 per passar a la següent. En cada iteració del bucle, s'havia de comprovar si *tablero[diagonal2]*, *tablero[diagonal2 + BSIZ - 1]* i *tablero[diagonal1 + 2*BSIZ - 2]* eren el mateix element. En cas afirmatiu, un jugador hauria fet tres en ratlla i la funció retornaria 'True'.

Vam haver d'implementar-ho d'aquesta forma, tot i que som conscients que en un taulell 3x3 no faria falta que sigués tant complicada, per poder adaptar el joc a qualsevol dimensió de taulell amb la que volguem jugar.

### stones():

```
def stones():
    tab=[]
    for i in range (0,len(tablero)):

        if tablero[i]=='X ':
            color = PLAYER_COLOR[0]
        elif tablero[i]=='O ':
            color = PLAYER_COLOR[1]
        else:
            color = GRAY
        j=0

        while j<len(tablero):
            if j<=i<j+BSIZ:
                tab += [(i//BSIZ,i-j,color)]
            j += BSIZ

    return tab
```

Aquesta funció, havia de retornar un iterable que contingués tuples de la forma: (fila, columna, color); amb totes les posicions del taulell. D'aquesta manera, vam fer una llista *tab* a la qual afegiriem totes les tuples corresponents a cada posició del taulell iterant sobre la llista *tablero* amb la variable *i*. La Paula ve pensar que seria millor, primer de tot, saber quin color tindrien i ja després mirar quina posició tenen. Per buscar el color, va haver de comprovar si hi havia *'X '*, *'O '* o *' '* a aquella posició i assignar-li el color corresponent al jugador utilitzant condicionals. Quan no hi havia cap jugador, el color era *GRAY*. Després, la Nadia va fer la part de buscar les coordenades dins del taulell. Per fer-ho, va utilitzar un while amb una variable *j* que recorregués la longitud de *tablero*. La *j* representava l'inici d'una línia del taulell, per això en cada iteració se li sumava BSIZ. L'objectiu era trobar en quina fila es troba la *i*. Quan ho trobàvem, afegíem a *tab* la tupla que contenia *i//BSIZ* (fila en la que es trobava), *i-j* (columna on es trobava) i *color* (color assignat al jugador o als espais buits).

### 1.2. Versió ampliada

Un cop vam tenir la versió bàsica implementada vam començar a pensar millores i canvis pel nostre programa. Aquestes modificacions es troben explicades en aquest apartat.

*i) abs_board_h*

En aquesta versió la variable 'cont' ja no ens era de la mateixa utilitat com a la versió bàsica, per tant, vam decidir que lo millor seria prescindir d'aquesta variable i la vam substituir per tres variables:

- La variable 'tur' la qual ens anirà indicant de qui és el torn (si del jugador 1 o del jugador 2).
- La variable 'tiradas_p1' la qual ens ajuda a saber si el jugador 1 ja ha utilitzat o no totes les seves pedres.
- La variable 'tiradas_p2' la qual ens ajuda a saber si el jugador 2 ja ha utilitzat o no totes les seves pedres.

***contador():***

```
def contador():
    nonlocal tur
    tur = 1- tur
    p = False
    if tur == 0:
        if tiradas_p1 < ST_PLAYER:
            p = True
    else:
        if tiradas_p2 < ST_PLAYER:
            p = True
    return p
```

Vam decidir crear la funció 'contador' per tal de poder portar a terme un seguiment de les tirades dels jugadors.

Aquesta funció el que fa és agafar el valor 'tur' que retorna la funció 'turno()' i aquest és restat al número 1 (per tal de poder canviar el torn a l'altre jugador). Un cop canviat el valor de la variable 'tur' el que es fa és mirar si el jugador associat al valor d'aquell instant de 'tur' ha fet menys tirades que quantitat de pedres té. En cas que es compleixi la variable 'p' pendrà el valor de 'True' si no és el cas 'p' serà False.

***turno():***

```
def turno():
    return tur
```

La funció 'turno' l'únic que fa és retornar tur per tal de saber a qui li toca tirar. Els únics valors que pot prendre tur són '0' o '1'. Si la variable tur val 0 significa que és el torn del jugador1 i si val 1 és el torn del jugador2.

## win():

```
def win():
    nonlocal tur
    if MODE == 0:
        print('',colored(PLY1, PLY1_C) if turno() == 1 else colored(PLY2, PLY2_C), 'won!\n', colored(PLY2, PLY2_C) if turno() == 1 else colored(PLY1, PLY1_C),"next time be more aware, don't give up!")
    else:
        print('',colored(PLY1, PLY1_C) if turno() == 0 else colored(PLY2, PLY2_C), 'won!\n', colored(PLY2, PLY2_C) if turno() == 0 else colored(PLY1, PLY1_C),"next time be more aware, don't give up!")
```

Vam decidir crear una funció anomenada 'win' per tal de poder indicar quin jugador havia guanyat la partida (dpenent del 'mode' triat pels jugadors el guanyador serà el jugador que ha aconseguit fer els tres en ratlla o el que no).

Aquesta funció l'únic que fa és imprimir per pantalla, quan el programa 'main_txt' o el programa 'main_gui' criden a 'win()', un missatge que indica qui és el guanyador i dona ànims a l'altre jugador.

### *ii) main_txt*

***Configuració de la partida:***

Per tal de poder fer la partida el més interactiva possible vam decidir que els jugadors poguessin configurar la partida al seu gust.

Els aspectes que vam decidir que els jugadors poden escollir:

- La mesura del taulell (BSIZ):

```
# choose the board size
BSIZ = int(input("\033[1;4mSelect BSIZ:\033[0m "))
while BSIZ < 3:
    print("BSIZ should be at least 3.")
    BSIZ = int(input("\033[1;4mSelect BSIZ:\033[0m "))
```

Per tal que l'usuari pogués escollir la mesura del taulell el que hem fet és que el codi mostri per pantalla el missatge 'Select BSIZ' per tal que l'usuari pugui introduir la mesura del taulell (i aquest valor que l'usuari ha introduït és el pren la variable BSIZ en el codi abs_board_h, ja que ara BSIZ ja no s'importa de 'constants', sinó que l'usuari és que ho podrà establir).

En el cas que l'usuari introdueixi un valor menor que 3 (tres és el valor mínim amb el qual es podria fer tres en ratlla) el codi imprimirà per pantalla 'BSIZ should be at least 3.' i es tornaria a demanar a l'usuari que introduis el valor de BSIZ una i una altra vegada fins introduir un valor vàlid per poder construir el taulell.

- La quantitat de pedres per jugador:

```
# choose the stones/player
ST_PLAYER = int(input("\033[1;4mSelect the amount of stones:\033[0m "))
while ST_PLAYER < 3 or ST_PLAYER > (BSIZ**2 - 1)//2:
    if ST_PLAYER < 3:
        print("The amount of stones should be at least 3.")
        ST_PLAYER = int(input("\033[1;4mSelect the amount of stones:\033[0m "))
    elif ST_PLAYER > (BSIZ**2 - 1)//2:
        print("Invalid amount of stones.")
        ST_PLAYER = int(input("\033[1;4mSelect the amount of stones:\033[0m "))
```

Per tal de que l'usuari pogués escollir quantes pedres volia tenir el que hem fet és bastant similar al tros de codi anterior, però aquest cas és demana a l'usuari introduir la quantitat de pedres que vol i si el valor introduït no és vàlid (és menor que tres, no seria possible fer tres en ratlla, o supera la quantitat màxima de pedres que podria haver al taulell, el total de caselles menys una entre dos) es fa saber a l'usuari que hi ha aquest problema i es torna a preguntar fins que el valor introduït sigui vàlid.

- El mode de joc:

```
# choose the mode
print("\n\033[1;4mChoose mode:\033[0m")
MODE = int(input("\033[1mClassic mode(0)\n\033[1mInverse mode(1)\n\033[0m"))
while MODE > 1 or MODE < 0:
    print("Invalid mode.")
    MODE = int(input("\033[1;4mChoose mode:\nClassic mode(0)\nInverse mode(1)\n\033[0m"))
```

En la mateixa línia que les anteriors configuracions, aquesta el que pregunta és si és vol jugar en mode classic, és a dir, que el guanyador sigui l'usuari que ha fet el tres en ratlla o el mode inverse on l'objectiu és evitar fer el tres en ratlla, per tant, el guanyador en aquest cas seria qui no aconseguis fer

els tres en ratlla. Segons el mode escollit pel usuari, aquest hauria d'introduir '1' o '0', la funció win() al final de la partida mostrarà qui és el guanyador.

Com en els anteriors casos, si el valor introduït per l'usuari no és vàlid és farà a l'usuari coneixedor d'aquest fet i se li tornarà a preguntar fins que introdueixi un valor vàlid.

- La dificultat de la partida:

```
# choose the difficulty
print("\n\033[1;4mChoose difficulty:\033[0m")
DIF = int(input("\033[1mSum mode(0)\nMultiplication mode(1)\n\033[0m"))
while DIF > 1 or DIF < 0:
    print("Invalid mode.")
    DIF = int(input("\033[1;4mChoose difficulty:\nSum mode(0)\nMultiplication mode(1)\n\033[0m"))
```

També vam pensar que seria bona idea que el nostre tres en ratlla implementat anés una mica més enllà d'un simple tres en ratlla on haguesis sols d'introduir les coordenades d'on posar la teva pedra, sinò que abans haguessis de superar un 'repte' per tal de poder tirar i que hi haguessin diferents nivells de complexitat del repte.

En el nivell de dificultat 'Sum mode', que seria el mode de menys dificultat, cal respondre correctament a una suma i en el nivell 'Multiplication mode', que seria el mode de màxima dificultat, cal respondre correctament a una multiplicació.

En aquesta part del codi l'usuari ha de seleccionar el nivell de dificultat que vol introduint '0' pel 'Sum mode' i '1' pel 'Multiplication mode'. En el cas d'introduir un valor diferent a aquests mencionat, se li farà saber a l'usuari que el mode que ha introduït no és vàlid i li farà tornar a introduir un altre cop el valor (fins que el valor introduït sigui vàlid).

- El nom/'nickname' i el color de cada jugador :

```
# conf your player
PLY1 = str(input("\n\033[1mName player1:\033[0m "))
print("\033[1;4mChoose your colour:\033[0m")
PLY1_C = str(input("\033[1mred,green,yellow,blue,magenta,cyan\n\033[0m"))
while PLY1_C != 'red' and PLY1_C != 'green' and PLY1_C != 'yellow' and PLY1_C != 'blue' and PLY1_C != 'magenta' and PLY1_C != 'cyan':
    print("Invalid colour.")
    PLY1_C = str(input("\033[1mred,green,yellow,blue,magenta,cyan\n\033[0m"))

PLY2 = str(input("\n\033[1mName player2:\033[0m "))
print("\033[1;4mChoose your colour:\033[0m")
PLY2_C = str(input("\033[1mred,green,yellow,blue,magenta,cyan\n\033[0m"))
while PLY2_C != 'red' and PLY2_C != 'green' and PLY2_C != 'yellow' and PLY2_C != 'blue' and PLY2_C != 'magenta' and PLY2_C != 'cyan' or PLY2_C== PLY1_C:
    if PLY2_C== PLY1_C:
        print("The colour you have chosen is already used for player1.")
        PLY2_C = str(input("\033[1mred,green,yellow,blue,magenta,cyan\n\033[0m"))
    else:
        print("Invalid colour.")
        PLY2_C = str(input("\033[1mred,green,yellow,blue,magenta,cyan\n\033[0m"))

print('\n',colored(PLY1,PLY1_C),"\033[1myou are\033[0m", colored("\033[1mX\033[0m", PLY1_C),'\n',colored(PLY2,PLY2_C),"\033[1myou are\033[0m", colored("\033[1mO\033[0m", PLY2_C))
```

Altrament, vam considerar que la propia personalització de l'usuari seria clau. Per tant, vam decidir que abans de començar la partida seria una bona idea que els usuaris poguessin escollir el seu 'nickname' per la partida i el color que prendran les seves pedres.

Per poder fer que les pedres siguessin del color desitjat per l'usuari vam haver d'importar el mòdul 'coloured' de python (pot ser que si aquest mòdul no es troba en l'ordinador en el qual s'està executant prèviament caldrà instal·lar-lo amb la comanda 'pip install coloured').

El que fa el codi és primer preguntar pel nom del primer jugador, mostrant per pantalla el missatge 'Name player1:' per tal que l'usuari introdueixi el seu nom i un cop aquest jugador hagi introduït el seu nom se li preguntarà per quin color vol ser, fent-li saber les opcions de color hi ha, per tal que

introdueixi un color vàlid i existent en el mòdul ‘coloured’. En el cas que no n’introdueixi cap d’aquests se li tornarà a preguntar de forma indefinida fins que n’introdueixi un de correcte.
Pel següent jugador es fa de la mateixa forma, però a l’hora d’escollir color a més d’haver d’introduir un color vàlid aquest no pot estar seleccionat per l’altre jugador.

### *explicació partida:*

```
# explanation of your game configuration
if MODE == 0 and DIF == 0:
    print("\n To win this game you have to make three in a row, but before you roll, you must respond correctly to a sum, if you don't answer well you will lose your turn.")
elif MODE == 1 and DIF == 0:
    print("\n To win this game you have to avoid making three in a row, but before rolling, you must answer correctly to a sum, if you don't answer correctly you will lose your turn.")
elif MODE == 1 and DIF == 1:
    print("\n To win this game you have to avoid making three in a row, but before rolling, you must answer correctly to a multiplication, if you don't answer correctly you will lose your turn.")
elif MODE == 0 and DIF == 1:
    print("\n To win this game you have to make three in a row, but before you roll, you must respond correctly to a multiplication, if you don't answer well you will lose your turn.")
input("\n\033[1mpress <enter> to start playing.\033[0m")
```

Un cop la configuració de la partida ja està feta pels usuaris es mostra pel terminal de quina forma ha estat configurada la partida, és a dir, s’explica que cal fer per guanyar la partida i que cal fer abans per poder col·locar una pedra.

### *while not stone_selected:*

```
while not stone_selected:
    print(PLY2 if turno()==1 else PLY1,end=", ")
    i, j = input("\033[1mselect stone coordinates: \033[0m").split()
    stone_selected = select_st(int(i), int(j))
    draw_txt(end)
```

En aquest tros de codi la única petita modificació que s’ha fet ha estat indicar de qui és el torn.

### *while stone_selected and not end:*

```
while stone_selected and not end:
    print("\033[1mIt's the turn of\033[0m",(colored(PLY2, PLY2_C) if turno()==1 else colored(PLY1, PLY1_C)))
    if DIF==0:
        x = randrange(0,100)
        y = randrange(0,100)
        print(x,'+',y,end=" ")
        res = int(input('= '))
    else:
        x = randrange(0,31)
        y = randrange(0,31)
        print(x,'x',y,end=" ")
        res = int(input('= '))

    if (DIF==0 and res == x+y) or (DIF==1 and res ==x*y):
        i, j = input("\033[1mSelect destination coordinates: \033[0m").split()
        stone_selected, curr_player, end = move_st(int(i), int(j))
        draw_txt(end)
    else:
        print("\nYou have lost your turn, the correct answer is", (x+y if DIF==0 else x*y))
        input("\033[1mpress <enter> to continue.\033[0m\n")
        stone_selected=contador()
```

Aquesta funció, s’ha modificat de la original per tal de poder introduir la configuració de la dificultat mencionada anteriorment.
Sota el taulell es mostrarà de qui és el torn, per tal de fer-ho més fàcil pels jugadors de saber a qui li toca tirar, ja que si aquests fallen la suma o la multiplicació (depenent de la dificultat que hagin escollit anteriorment) es perd el torn, pot arribar a ser una mica enrevessat enrecordar-se a qui li toca tirar.
Sota aquest missatge ja es trobaria l’operació a resoldre pel jugador que li toca tirar, en el cas de respondre-la bé se li deixaria introduir les coordenades on vol posar la seva pedra i en cas contrari se li mostraria per pantalla el missatge ‘You have lost your turn, the correct answer is ’ amb el resultat correcte de l’operació. Seguidament caldria clicar la tecla ‘enter’ per tal de poder seguir la partida, la qual continuarà amb el torn de l’altre jugador amb la seva respectiva operació a resoldre.

Per crear les operacions i que aquestes no sempre siguessin les mateixes s'ha importat el mòdul 'random' de python. D'aquesta forma hem pogut utilitzar la funció de randrange per tal que cada operant sigués un número aleatori en el rang (0,100) per la suma i en el rang (0,31) per la multiplicació.

***Final:***

```
time.sleep(0.5)
print("\033[1mGame over.\033[0m")
win()
```

En la versió bàsica del programa, un cop la partida era finalitzada sols es mostrava per pantalla el missatge 'Game over'. Nosaltres vam decidir que seria millor que a més d'indicar que la partida havia acabat que mostres qui havia sortit el/la guanyador/a de la partida, per tant, sota el missatge 'Game over' hem posat que es retorni la funció 'win()' (explicada anteriorment) creada en el fitxer 'abs_board_h'.

També vam decidir importar 'time' de python per tal que abans que es mostri que la partida ha finalitzat i qui és el guanyador s'esperi mig segon per a què els jugadors puguin adonar-se que ha hagut un tres en ratlla.

### iii) main_gui

En la versió del 'main_gui' també vam voler seguir en la mateixa línia que amb el 'main_txt', però aquest cop al executar-se en el pygame enlloc de fer-ho en el terminal hi ha aspectes de la configuració de la partida que abans es podien fer que ara no. A continuació es mostren les implementacions afegides en aquesta versió gràfica.

#### *Configuració de la partida:*

```
BSIZ = int(input("\033[1;4mSelect BSIZ:\033[0m "))
while BSIZ < 3:
    print("BSIZ should be at least 3.")
    BSIZ = int(input("\033[1;4mSelect BSIZ:\033[0m "))

ST_PLAYER = int(input("\033[1;4mSelect the amount of stones:\033[0m "))
while ST_PLAYER < 3 or ST_PLAYER > (BSIZ**2 - 1)//2:
    if ST_PLAYER < 3:
        print("The amount of stones should be at least 3.")
        ST_PLAYER = int(input("\033[1;4mSelect the amount of stones:\033[0m "))
    elif ST_PLAYER > (BSIZ**2 - 1)//2:
        print("Invalid amount of stones.")
        ST_PLAYER = int(input("\033[1;4mSelect the amount of stones:\033[0m "))

print("\n\033[1;4mChoose mode:\033[0m")
MODE = int(input("\033[1mClassic mode(0)\nInverse mode(1)\n\033[0m"))
while MODE > 1 or MODE < 0:
    print("Invalid mode.")
    MODE = int(input("\033[1;4mChoose mode:\nClassic mode(0)\nInverse mode(1)\n\033[0m"))

PLY1 = str(input("\n\033[1mName player1:\033[0m "))
PLY2 = str(input("\033[1mName player2:\033[0m "))

PLY1_C = str('blue')
PLY2_C = str('red')

if MODE == 0:
    print("To win this game you have to make three in a row.")
elif MODE == 1:
    print("To win this game you have to avoid making three in a row.")
input("\n\033[1mpress <enter> to start playing.\033[0m")
```

Les funcions per seleccionar la mesura del taulell (BSIZ), la quantitat de pedres per jugador, l'elecció de mode de partida estan implementades de la mateixa forma que ho estan en el fitxer 'main_txt'.

D'altra banda, per tal de configurar el jugador, tan sols es demana per introduir el 'nickname' de cada jugador, ja que amb el pygame ja venen definits els colors dels jugadors.

Finalment, és mostra també el missatge explicatiu de com ha quedat configurada la partida segons les eleccions fetes pels jugadors.

#### *Final:*

```
# Friendly finish-up:
print('Game over')
win()

pygame.quit()
```

Un cop s'ha fet el tres en ratlla pel terminal es mostra qui ha guanyat amb la funció 'win()'.

## 2. DIFICULTATS PRINCIPALS

**Comprensió de la tasca:**

En un principi, ens va costar entendre la funcionalitat de cada funció i la implementació d'aquestes (què havien de retornar) dins dels programes main_txt.py i main_gui.py. A més, quan intentàvem executar el programa main_txt.py ens donaven errors per tabulació (Indent error) els quals ens va costar identificar.

**Distinció columna-fila:**

Quan vam començar a programar, ens vam plantejar que la quantitat de files i columnes del taulell podrien ser diferents, és a dir, que el taulell no fos necessàriament quadrat. Això ens va portar problemes a l'hora de seleccionar les caselles correctes. Després, ens vam adonar que la grandària el taulell venia definida per la constant 'BSIZ', per tant, el taulell sempre era quadrat de dimensió BSIZ x BSIZ. Això ens va simplificar molt a l'hora de determinar les caselles.

**Implementació del main_gui.py:**

Un cop el main_txt.py ens funcionava, vam provar l'execució del main_gui.py. Com que el main_txt.py no feia servir la funció stones() ni la variable 'curr_player', no les vam tenir en compte en un primer moment. Quan ens vam adonar que al main_gui.py sí feien falta i vam entendre quins valors els hi havíem de donar, vam modificar el abs_board_h.py per tal que funcionés correctament.

**Contador (versió ampliada):**

En la versió ampliada, no ens va ser útil per determinar a quin jugador li tocava jugar ni quantes pedres havien jugat cadascun, ja que si es fallava la pregunta, el contador augmentava i el jugador no havia fet cap moviment. Per solucionar aquest problema, vam introduir 3 noves variables: tur (per saber a qui li tocava jugar), tiradas_p1 (per portar un seguiment de les tirades de jugador 1) i tiradas_p2 (per portar un seguiment de les tirades de jugador 2).

# 3. OBJECTIUS ASSOLITS

**Posar en pràctica els coneixements assolits**

Hem volgut que en les implementacions del programa es veiés reflectit tot allò après durant les classes de programació i que no havíem pogut fer servir abans com: importar llibreries (random i termcolor), utilitzar el 'nonlocal', fer servir la funció randrange...

A més, també hem pogut posar en pràctica llistes, iteracions amb 'while', iteracions amb 'for', condicionals, funcions locals... en un programa no únicament centrat en una funció, sinó que ho engloba tot.

**Aconseguir una versió alternativa i original**

Ens vam proposar fer una versió del programa que anés una mica més enllà d'un simple 3 en ratlla, que per poder aconseguir la victòria s'hagués de superar un repte. Vam considerar que realitzar un càlcul (més o menys senzill) mentalment podria ser una bona forma per millorar les habilitats de càlcul mental dels jugadors.

A més, voliem que la nostra proposta sigui el més personalitzada possible per l'usuari el qual pot escollir la mesura del taulell, el nombre de pedres, el mode de joc, la dificultat, el seu nom i el color de les seves pedres.

# 4. VALORACIÓ DE L'APRENENTATGE

Aquesta experiència ha estat enriquidora i ha contribuït significativament en el nostre creixement com a programadores. Aquestes són algunes reflexions sobre l'aprenentatge obtingut:

- Col·laborar conjuntament dues companyes de classe en aquest projecte va ser una oportunitat única per comprendre la importància del treball en equip en el món de la programació. Aquesta dinàmica va requerir comunicació constant i va millorar les nostres habilitats per treballar en equip.
- La creació del tres en ratlla ens ha plantejat desafiaments interessants, des de la lògica del joc fins a totes les variants fetes. Aprendre a resoldre aquests problemes de manera eficaç ha millorat la nostra capacitat per abordar reptes de programació.

En conjunt, el treball de programació per parelles en el desenvolupament del joc "3 en Ratlla" ha estat una oportunitat educativa significativa. A més d'adquirir coneixements tècnics, hem après a treballar en equip, afrontar reptes amb confiança i aplicar mètodes apresos al llarg de les classes de programació.